

TÀI LIỆU HƯỚNG DẪN THỰC HÀNH

LAB 01 - RAG FOUNDATION

Xử Lý Tài Liệu, Lập Chỉ Mục Vector và Xây Dựng Pipeline Hỏi-Đáp Căn Bản Với Gemini

Trần Hồng Việt¹ - Vũ Minh Sơn¹ - Nguyễn Tiến Khôi¹

¹Đại học Công nghệ - Đại học Quốc Gia Hà Nội (UET-VNU)

Thông tin bài học

- **Mục tiêu:** Nắm vững vòng đời của một hệ thống RAG cơ bản: từ tiếp nhận tài liệu đa định dạng, phân đoạn (chunking), tạo vector embedding, lưu trữ chỉ mục trên Vector Database đến truy xuất ngữ cảnh và sinh câu trả lời có trích dẫn nguồn (Citation).
- **Tech Stack:** Python 3.11+, Gemini API (gemini-flash-lite-latest, text-embedding-004), ChromaDB / Pinecone, PyMuPDF, python-docx, pandas, pytesseract.
- **Mã nguồn áp dụng:** Các module trong 01_rag_foundation/ và rag_core/.

MỤC LỤC

I. VÌ SAO CẦN ĐẾN RAG?	3
🔗 So sánh RAG (Retrieval-Augmented Generation) và Fine-tuning	3
II. BỨC TRANH TỔNG QUAN: RAG HOẠT ĐỘNG NHƯ THẾ NÀO?	4
III. GIAI ĐOẠN 1 - INGESTION, CHUNKING & EMBEDDING	4
1. Tiếp nhận và nắn chuẩn tài liệu đa định dạng (rag_core/document_processing.py)	4
2. Chiến lược phân đoạn (Chunking Strategies)	4
IV. GIAI ĐOẠN 2 - QUẢN LÝ VECTOR STORE (CHROMADB & PINECONE)	5
V. GIAI ĐOẠN 3 - RETRIEVAL & GROUNDED GENERATION	5
VI. THỰC HÀNH: MÃ NGUỒN TRIỂN KHAI HOÀN CHỈNH (COPY-PASTE RUNS 100%)	5
🔗 Bước 0: Chuẩn bị Môi trường & Tập .env	5
📄 Script 1: Phân đoạn và Lập chỉ mục Vector (01_rag_foundation/01_chunk_and_index.py)	6
📄 Script 2: Truy xuất Vector & Sinh Câu Trả Lời (01_rag_foundation/02_retrieve_and_answer.py)	8
📄 Script 3: Đồng bộ Vector Store độc lập (01_rag_foundation/03_sync_vector_store.py)	9
Script 4: Mã Python Thuần Tương Tác Trực Tiếp (Copy & Run trong Jupyter Notebook / Python File)	10
VII. PHÂN TÍCH MÃ NGUỒN CỐT LÕI (rag_core/)	12
1. build_or_load_index() trong rag_core/runtime.py	12
2. sync_records() trong rag_core/vector_store.py	12
VIII. XỬ LÝ LỖI THƯỜNG GẶP (TROUBLESHOOTING)	12
IX. CÂU HỎI TRẮC NGHIỆM Củng Cố Kiến Thức	13
PHỤ LỤC: ĐÁP ÁN & GIẢI THÍCH CHI TIẾT	16
TÀI LIỆU THAM KHẢO	17

I. VÌ SAO CẦN ĐẾN RAG?

Các mô hình ngôn ngữ lớn (LLM) hiện đại như Google Gemini, GPT-4 có khả năng suy luận vượt trội, nhưng khi áp dụng vào môi trường doanh nghiệp hoặc thực tế giảng dạy, chúng gặp phải 3 rào cản cốt lõi:

1. **Kiến thức bị đóng băng (Knowledge Cutoff):** Mô hình chỉ "biết" những thông tin đã được đưa vào tập dữ liệu huấn luyện tại thời điểm tạo mô hình.
2. **Hiện tượng Ảo giác (Hallucination):** Khi được hỏi về những thông tin mà mô hình không biết hoặc không rõ, LLM có xu hướng tự "bịa" ra câu trả lời nghe rất thuyết phục nhưng sai sự thật.
3. **Không tiếp cận được dữ liệu nội bộ/bảo mật:** Các tài liệu quy trình, văn bản pháp lý, hợp đồng hay hồ sơ khách hàng hoàn toàn nằm ngoài dữ liệu công khai trên Internet.

So sánh RAG (Retrieval-Augmented Generation) và Fine-tuning

Tiêu chí	Fine-tuning	RAG (Retrieval-Augmented Generation)
Cơ chế cập nhật	Huấn luyện lại trọng số (weights) của LLM dựa trên tập dữ liệu mới.	Giữ nguyên trọng số LLM; tìm kiếm ngữ cảnh liên quan và chèn vào Prompt.
Chi phí tính toán	Rất cao: Cần nhiều GPU đắt tiền (A100/H100) và thời gian huấn luyện.	Rất thấp: Chỉ tốn chi phí gọi API Embedding & LLM Inference.
Cập nhật dữ liệu mới	Phải tốn thời gian huấn luyện lại (Retrain) mỗi khi có tài liệu mới.	Tức thì: Chỉ cần lưu tài liệu mới vào Vector DB.
Kiểm soát Ảo giác	Khó kiểm soát; mô hình vẫn có thể bịa thông tin từ trọng số.	Rất cao: Ép mô hình chỉ trả lời dựa trên ngữ cảnh được cung cấp.
Khả năng trích dẫn	Không thể trích dẫn chính xác trang/tệp nguồn.	Rõ ràng: Có thể truy xuất chính xác tên file, trang, vị trí đoạn văn (Citation).

II. BỨC TRANH TỔNG QUAN: RAG HOẠT ĐỘNG NHƯ THẾ NÀO?

Một hệ thống RAG cơ bản gồm 3 giai đoạn độc lập nhưng kết nối chặt chẽ với nhau:

```

flowchart TD
    subgraph Giai_đoạn_1 [Giai đoạn 1: Indexing]
        A[Tài liệu thô: PDF, DOCX, MD, OCR] --> B[Nắn chuẩn Unicode & Clean Text]
        B --> C[Phân đoạn - Chunking]
        C --> D[Gọi Gemini Embedding 768-dim]
        D --> E[Lưu Storage JSON & Vector DB]
    end

    subgraph Giai_đoạn_2 [Giai đoạn 2: Retrieval]
        F[Người dùng nhập câu hỏi] --> G[Gemini Embedding cho Query]
        G --> H[Vector Search: Cosine Similarity]
        E --> H
        H --> I[Lấy Top-K Chunks liên quan nhất]
    end

    subgraph Giai_đoạn_3 [Giai đoạn 3: Generation]
        I --> J[Tạo Grounded Prompt + Context]
        F --> J
        J --> K[LLM Gemini Flash]
        K --> L[Câu trả lời + Trích dẫn nguồn Citation]
    end
  
```

III. GIAI ĐOẠN 1 - INGESTION, CHUNKING & EMBEDDING

1. Tiếp nhận và nắn chuẩn tài liệu đa định dạng (rag_core/document_processing.py)

Trong thực tế, tài liệu bao gồm PDF, Word (.docx), Excel (.xlsx) và cả ảnh scan (.png, .jpg).

Module document_processing.py thực hiện:

- **Chuẩn hóa Unicode tiếng Việt:** Đưa toàn bộ ký tự về dạng NFC, xóa khoảng trắng thừa và dọn dẹp các ký tự ẩn bằng `unicodedata.normalize("NFC", text)`.
- **Trích xuất thông minh theo trang/sheet:**
 - Tập .pdf: PyMuPDF (fitz) trích xuất từng trang.
 - Tập .docx: python-docx trích xuất đoạn văn và bảng.
 - Tập .xlsx: pandas chuyển từng sheet thành văn bản.
 - Tập hình ảnh: pytesseract OCR với `lang="vie+eng"`.

2. Chiến lược phân đoạn (Chunking Strategies)

Tài liệu được chia thành các đoạn nhỏ (chunks) nhờ hàm `chunk_document()`:

4. **Fixed-size Chunking:** Cắt cố định theo dung lượng ký tự (`chunk_size=1400`, `overlap=220`).
5. **Semantic Chunking:** Cắt dựa theo ranh giới đoạn văn (`\n\n+`).
6. **Hierarchical Chunking (Default):** Nhận diện tiêu đề pháp lý (Điều X, Chương Y, # Heading) để gán `parent_title` và `parent_id`.

IV. GIAI ĐOẠN 2 - QUẢN LÝ VECTOR STORE (CHROMADB & PINECONE)

Vector Store đóng vai trò lưu trữ các vector 768 chiều và thực hiện tìm kiếm Cosine Similarity:

- **chroma (ChromaDB):** Vector DB chạy local/Docker (`hnsw:space: cosine`). Cổng kết nối: `http://localhost:8001`.
- **pinecone:** Vector DB Cloud.
- **json:** Backend fallback offline lưu tệp `storage/foundation.json`.

V. GIAI ĐOẠN 3 - RETRIEVAL & GROUNDED GENERATION

Khi người dùng đặt câu hỏi Q :

7. Q được tạo vector embedding bằng Gemini API.
8. ChromaDB tìm Top-K ($K = 5$) chunk có độ tương đồng Cosine cao nhất.
9. LLM sinh câu trả lời và ép trích dẫn nguồn dạng `[tên_file, chunk_id]`.

VI. THỰC HÀNH: MÃ NGUỒN TRIỂN KHAI HOÀN CHỈNH

Phần này cung cấp mã nguồn **ĐẦY ĐỦ 100% (Full Standalone Code)**. Học viên có thể copy từng block code tạo thành file `.py` hoặc chạy trực tiếp trong VS Code / Jupyter Notebook mà **KHÔNG** bị thiếu bất kỳ import hay tham số nào.

Bước 0: Chuẩn bị Môi trường & Tệp .env

1. Tạo môi trường ảo và cài đặt thư viện (PowerShell):

```
python -m venv .venv
.\venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
python -m pip install -r requirements.txt
```

2. Khởi tạo tệp cấu hình môi trường .env tại D:\RAG_Demo_Labs-main\.env:

```
GEMINI_API_KEY=AIzaSy...Thay_API_Key_Cua_Ban_Vao_Day
GEMINI_MODEL=gemini-flash-lite-latest
RAG_VECTOR_BACKEND=chroma
CHROMA_URL=http://localhost:8001
```

3. Khởi động ChromaDB Server bằng Docker:

```
docker compose up -d chromadb
```

Script 1: Phân đoạn và Lập chỉ mục Vector (01_rag_foundation/01_chunk_and_index.py)

Tập mã nguồn hoàn chỉnh 100% dùng để đọc tài liệu trong data/, chia chunk, tạo Gemini Embedding và đồng bộ sang ChromaDB:

```
#
=====
=====
# File: 01_rag_foundation/01_chunk_and_index.py
# Mô tả: Giai đoạn 1 - Parsing tài liệu, Chunking, Gemini Embedding và Indexing
#
=====
=====

import sys
import argparse
from pathlib import Path

# Thiết lập đường dẫn tới thư mục gốc dự án
HERE = Path(globals().get("__file__", Path.cwd())).resolve()
PROJECT_ROOT = HERE.parents[1] if HERE.parent.name == "01_rag_foundation"
else HERE
sys.path.insert(0, str(PROJECT_ROOT))

# Import các hàm cốt lõi từ rag_core
from rag_core.runtime import build_or_load_index, chunk_document, markdown_files
```

```

from rag_core.vector_store import sync_records

def main():
    parser = argparse.ArgumentParser(description="Giai đoạn 1: Chia tài liệu và tạo chỉ mục embedding Gemini")
    parser.add_argument("--rebuild", action="store_true", help="Bắt buộc tạo lại chỉ mục từ đầu")
    parser.add_argument("--dry-run", action="store_true", help="Hiển thị kế hoạch chia đoạn mà không gọi Gemini API")
    parser.add_argument("--backend", choices=["chroma", "pinecone", "json"], default=None, help="Vector backend (mặc định lấy từ RAG_VECTOR_BACKEND trong .env)")
    args = parser.parse_args()

    # Nếu chạy chế độ xem trước kế hoạch (--dry-run)
    if args.dry_run:
        plan = {path.name: len(chunk_document(path)) for path in markdown_files()}
        print("=== KẾ HOẠCH CHIA ĐOẠN (DRY RUN) ===")
        for doc_name, count in plan.items():
            print(f" - {doc_name}: {count} chunks")
        print(f"Tổng cộng: {sum(plan.values())} chunks (Chưa gọi Gemini API)")
        raise SystemExit(0)

    # 1. Tạo chỉ mục hoặc nạp từ storage/foundation.json
    print("Đang tiến hành lập chỉ mục dữ liệu...")
    records = build_or_load_index("foundation", force=args.rebuild)

    # 2. Đồng bộ chỉ mục vector lên Vector DB (ChromaDB / Pinecone)
    sync_records(records, backend=args.backend)

    print("\n=== HOÀN TẤT LẬP CHỈ MỤC ===")
    print(f"✓ Đã lập chỉ mục thành công: {len(records)} đoạn văn")
    print(f"✓ Đồng bộ thành công lên Vector Backend: {args.backend or 'chroma (mặc định)'}")
    print(f"✓ Danh sách nguồn tài liệu:", sorted({r['source'] for r in records}))

if __name__ == "__main__":
    main()

```

Lệnh thực thi Script 1 trên Terminal:

10. Xem trước kế hoạch chunking (không gọi API):

```
python 01_rag_foundation/01_chunk_and_index.py --dry-run
```

11. Lập chỉ mục và nạp vào ChromaDB:

```
python 01_rag_foundation/01_chunk_and_index.py --rebuild --backend chroma
```

Script 2: Truy xuất Vector & Sinh Câu Trả Lời (01_rag_foundation/02_retrieve_and_answer.py)

Tập mã nguồn hoàn chỉnh 100% dùng để tìm kiếm ngữ cảnh Cosine Similarity trong ChromaDB và gửi prompt cho Gemini sinh câu trả lời:

```
#
=====
=====
# File: 01_rag_foundation/02_retrieve_and_answer.py
# Mô tả: Giai đoạn 2 - Truy xuất Vector Search và Sinh câu trả lời với Gemini
#
=====
=====

import sys
import argparse
from pathlib import Path

# Thiết lập đường dẫn tới thư mục gốc dự án
HERE = Path(globals().get("__file__", Path.cwd())).resolve()
PROJECT_ROOT = HERE.parents[1] if HERE.parent.name == "01_rag_foundation"
else HERE
sys.path.insert(0, str(PROJECT_ROOT))

# Import các hàm cốt lõi từ rag_core
from rag_core.runtime import load_index, retrieve, infer

def main():
    parser = argparse.ArgumentParser(description="Giai đoạn 2: Truy xuất từ chỉ mục
hoàn tất và gọi Gemini QA")
    parser.add_argument("--ask", required=True, help="Câu hỏi cần tra cứu")
    parser.add_argument("--backend", choices=["chroma", "pinecone", "json"],
default=None,
```



```

        help="Vector backend (chroma/pinecone/json)")
args = parser.parse_args()

# 1. Nạp chỉ mục đã lưu từ Lab 01
records = load_index("foundation")

# 2. Truy xuất Top-K (K=5) đoạn văn liên quan nhất
hits = retrieve(args.ask, records, top_k=5, backend=args.backend)

# 3. Hiển thị danh sách Top-K chunks được tìm thấy
print(f"\n=== KẾT QUẢ TRUY XUẤT TOP-{len(hits)} CHUNKS TƯƠNG ĐỒNG ===")
for hit in hits:
    print(f"Score: {hit['score']:.3f} | File: {hit['source']} | Chunk ID: {hit['chunk_id']}")

# 4. Gửi ngữ cảnh vào Gemini để sinh câu trả lời kèm Citation
print("\n=== CÂU TRẢ LỜI TỪ GEMINI LLM ===")
answer = infer(args.ask, hits)
print(answer)

if __name__ == "__main__":
    main()

```

Lệnh thực thi Script 2 trên Terminal:

```
python 01_rag_foundation/02_retrieve_and_answer.py --backend chroma --ask "Thông tin nào quy định về hoạt động cho vay?"
```

Script 3: Đồng bộ Vector Store độc lập (01_rag_foundation/03_sync_vector_store.py)

Tập mã nguồn hoàn chỉnh 100% dùng để đẩy chỉ mục từ *storage/foundation.json* sang *Pinecone Cloud* hoặc *ChromaDB*:

```

#
=====
=====
# File: 01_rag_foundation/03_sync_vector_store.py
# Mô tả: Đồng bộ chỉ mục Lab 01 sang ChromaDB hoặc Pinecone Cloud

```

```
#
=====

import sys
import argparse
from pathlib import Path

HERE = Path(globals().get("__file__", Path.cwd())).resolve()
PROJECT_ROOT = HERE.parents[1] if HERE.parent.name == "01_rag_foundation"
else HERE
sys.path.insert(0, str(PROJECT_ROOT))

from rag_core.runtime import load_index
from rag_core.vector_store import sync_records

def main() -> None:
    parser = argparse.ArgumentParser(description="Đồng bộ chỉ mục sang Vector
    Database")
    parser.add_argument("--backend", choices=["chroma", "pinecone"], required=True,
    help="Tên backend Vector DB")
    parser.add_argument("--collection", default="rag_foundation", help="Tên
    collection/namespace trong Vector DB")
    args = parser.parse_args()

    # Nạp chỉ mục và thực hiện đồng bộ
    records = load_index("foundation")
    count = sync_records(records, backend=args.backend, collection=args.collection)
    print(f"✓ Đã đồng bộ thành công {count} vector sang {args.backend}
    (Collection/Namespace: {args.collection})")

if __name__ == "__main__":
    main()
```

Script 4: Mã Python Thuần Tương Tác Trực Tiếp (Copy & Run trong Jupyter Notebook / Python File)

Học viên có thể copy nguyên khối mã nguồn dưới đây tạo thành file `demo_standalone_rag.py` và bấm RUN ngay lập tức không cần chờ dòng lệnh CLI:

```

#
=====

=====
# File: demo_standalone_rag.py (Dành cho chạy trực tiếp trên Jupyter / Python script)
# Mô tả: Chạy toàn bộ pipeline RAG Lab 01 trong 1 file Python duy nhất
#
=====

=====
import os
import sys
from pathlib import Path

# Thêm đường dẫn gốc dự án
PROJECT_ROOT = Path.cwd()
sys.path.insert(0, str(PROJECT_ROOT))

from rag_core.runtime import build_or_load_index, retrieve, infer
from rag_core.vector_store import sync_records

def run_rag_demo():
    print("--- 1. Đang tiến hành tạo chỉ mục và embedding dữ liệu ---")
    # Tự động nạp hoặc tạo lại index
    records = build_or_load_index("foundation", force=False)

    print("--- 2. Đồng bộ Vector vào ChromaDB ---")
    sync_records(records, backend="chroma")

    print("--- 3. Đặt câu hỏi thử nghiệm ---")
    question = "Thời hạn cơ cấu lại thời hạn trả nợ được quy định như thế nào?"
    print(f"Câu hỏi: {question}\n")

    print("--- 4. Tìm kiếm ngữ cảnh tương đồng (Similarity Search) ---")
    hits = retrieve(question, records, top_k=3, backend="chroma")
    for idx, hit in enumerate(hits, 1):
        print(f" [{idx}] Top Score: {hit['score']:.3f} | Nguồn: {hit['source']} (chunk {hit['chunk_id']})")
        print(f"    Nội dung: {hit['text'][:120]}...\n")

    print("--- 5. Gemini sinh câu trả lời kèm Trích dẫn nguồn ---")
    answer = infer(question, hits)
    print("KẾT QUẢ:")

```

```
print(answer)

if __name__ == "__main__":
    run_rag_demo()
```

VII. PHÂN TÍCH MÃ NGUỒN CỐT LỖI (rag_core/)

Để hiểu sâu cơ chế hoạt động, học viên cần nắm vững 3 hàm cốt lõi trong rag_core/:

1. build_or_load_index() trong rag_core/runtime.py

Tự động kiểm tra vân tay dữ liệu (fingerprint SHA-256). Nếu tệp trong data/ không thay đổi, hàm sẽ nạp lại ngay từ storage/foundation.json để tránh gọi lặp lại Gemini API.

2. sync_records() trong rag_core/vector_store.py

Thực hiện mã hóa nội dung nhạy cảm và đẩy dữ liệu lên ChromaDB / Pinecone qua phương thức upsert.

VIII. XỬ LÝ LỖI THƯỜNG GẶP (TROUBLESHOOTING)

Hiện tượng lỗi	Nguyên nhân	Cách khắc phục chuẩn xác
RuntimeError: Thiếu GEMINI_API_KEY	Chưa tạo tệp .env hoặc chưa điền API Key.	Kiểm tra tệp .env tại thư mục gốc, đảm bảo có dòng GEMINI_API_KEY=AIzaSy...
UnicodeEncodeError / Tiếng Việt bị lỗi chữ	Terminal Windows (PowerShell/CMD) dùng mã trang cũ (cp1252).	Chạy lệnh chcp 65001 trong PowerShell trước khi chạy Python để bật mã hóa UTF-8.
ChromaDB không kết nối (Connection Refused)	Container chromadb chưa chạy hoặc sai cổng 8001.	Chạy docker compose up -d chromadb và kiểm tra bằng docker compose ps.

Hiện tượng lỗi	Nguyên nhân	Cách khắc phục chuẩn xác
HTTP 429 Too Many Requests từ Gemini API	Vượt quá giới hạn Rate Limit của tài khoản Gemini miễn phí.	Hàm embed() trong runtime.py đã tích hợp time.sleep(0.25) và retry ngắt lùi (exponential backoff).
ValueError: Unsupported document type	File trong data/ không nằm trong danh sách hỗ trợ (.md, .pdf, .docx, .xlsx, .png).	Kiểm tra định dạng tệp đầu vào, chỉ bỏ các tệp hợp lệ vào thư mục data/.

IX. CÂU HỎI TRẮC NGHIỆM Củng Cố Kiến Thức

Câu 1. Vì sao trong hệ thống RAG doanh nghiệp, phương pháp RAG lại vượt trội hơn Fine-tuning khi cần cập nhật văn bản pháp lý mới ban hành?

- A. Vì Fine-tuning không thể xử lý được tiếng Việt.
- B. Vì RAG cập nhật kiến thức ngay lập tức chỉ bằng cách lưu tài liệu vào Vector DB mà không tốn chi phí huấn luyện lại mô hình.
- C. Vì Fine-tuning bắt buộc phải sử dụng phần cứng của Google.
- D. Vì RAG loại bỏ 100% thời gian phản hồi của LLM.

Câu 2. Tham số overlap trong kỹ thuật Chunking đóng vai trò cốt lõi gì?

- A. Giới hạn dung lượng tệp lưu trữ trên đĩa cứng.
- B. Lặp lại một đoạn nội dung cuối của chunk trước sang đầu chunk sau, giúp bảo toàn tính liên tục ngữ cảnh tại điểm cắt.
- C. Giúp tăng gấp đôi tốc độ tạo embedding của Gemini API.
- D. Tự động dịch văn bản sang tiếng Anh trước khi đưa vào Vector Store.

Câu 3. Mô hình embedding Gemini text-embedding-004 trong Lab 01 trả về vector có bao nhiêu chiều (dimensions)?

- A. 128 chiều.
- B. 512 chiều.
- C. 768 chiều.
- D. 1536 chiều.

Câu 4. Trong mã nguồn `rag_core/document_processing.py`, thư viện nào được sử dụng để trích xuất văn bản từ tệp PDF theo từng trang?

- A. pandas
- B. python-docx
- C. PyMuPDF (fitz)
- D. pytesseract

Câu 5. Khi người dùng chạy lệnh `python 01_rag_foundation/01_chunk_and_index.py --dry-run`, điều gì sẽ xảy ra?

- A. Toàn bộ dữ liệu được đẩy lên ChromaDB ngay lập tức.
- B. Hệ thống tính toán và in ra số lượng chunk của từng tệp tài liệu mà KHÔNG gọi Gemini API.
- C. Hệ thống xóa toàn bộ dữ liệu cũ trong thư mục `storage/`.
- D. Hệ thống tự động khởi chạy giao diện Web Streamlit.

Câu 6. Để tìm kiếm các vector có độ tương đồng cao nhất với câu hỏi trong ChromaDB, thước đo khoảng cách nào được cấu hình trong `vector_store.py`?

- A. Euclidean Distance (l2)
- B. Manhattan Distance
- C. Cosine Similarity (hnsw:space: cosine)
- D. Jaccard Index

Câu 7. Thuộc tính `task_type` nào được truyền vào Gemini API khi tạo embedding cho các đoạn tài liệu lưu vào chỉ mục?

- A. RETRIEVAL_QUERY
- B. RETRIEVAL_DOCUMENT

- C. CLASSIFICATION
- D. SEMANTIC_SIMILARITY

Câu 8. Hàm `infer()` trong `rag_core/runtime.py` ngăn chặn ảo giác (hallucination) bằng cơ chế nào?

- A. Đóng băng hoàn bộ bộ nhớ tạm của GPU.
- B. Đưa vào System Prompt yêu cầu LLM chỉ trả lời dựa trên ngữ cảnh được cung cấp và bắt buộc trích dẫn Citation.
- C. Bắt buộc người dùng nhập câu hỏi bằng tiếng Anh.
- D. Tự động từ chối tất cả các câu hỏi dài quá 10 từ.

Câu 9. Tệp `storage/foundation.json` trong dự án có vai trò gì?

- A. Là tệp cấu hình chứa API Key của Google Gemini.
- B. Lưu bản sao chỉ mục đoạn văn, metadata và vector embedding để tái sử dụng mà không phải gọi lại Gemini API khi tệp nguồn không đổi.
- C. Chứa giao diện mã nguồn của ứng dụng Web Chatbot.
- D. Lưu trữ mật khẩu đăng nhập của quản trị viên.

Câu 10. Trong môi trường Terminal Windows (PowerShell), nếu gặp lỗi hiển thị sai font chữ tiếng Việt, câu lệnh nào cần chạy trước khi thực thi lệnh Python?

- A. `docker compose stop`
- B. `chcp 65001`
- C. `pip install unicode`
- D. `python --version`

PHỤ LỤC: ĐÁP ÁN & GIẢI THÍCH CHI TIẾT

Câu	Đáp án	Giải thích chi tiết
1	B	RAG chỉ cần chèn văn bản mới vào Vector DB là LLM truy xuất được ngay, trong khi Fine-tuning phải huấn luyện lại trọng số rất tốn chi phí và thời gian.
2	B	Overlap giữ lại một phần ký tự của chunk trước để tránh việc câu văn/ý nghĩa bị ngắt đôi gây mất ngữ cảnh tại ranh giới cắt.
3	C	Theo cấu hình <code>output_dimensionality=768</code> trong <code>runtime.py</code> , mô hình Gemini text-embedding-004 xuất ra vector 768 chiều.
4	C	<code>document_processing.py</code> dùng PyMuPDF (<code>import fitz</code>) để đọc tệp PDF theo từng trang (<code>page.get_text("text")</code>).
5	B	Cờ <code>--dry-run</code> giúp kiểm tra số lượng chunk dự kiến mà không thực hiện gọi API embedding, giúp tiết kiệm chi phí và quota.
6	C	Cấu hình <code>metadata={"hnsw:space": "cosine"}</code> trong <code>_chroma_collection()</code> thiết lập không gian tìm kiếm Cosine Similarity.
7	B	Khi tạo embedding cho tài liệu lưu kho, <code>task_type</code> bắt buộc là <code>RETRIEVAL_DOCUMENT</code> . Khi tìm kiếm câu hỏi mới dùng <code>RETRIEVAL_QUERY</code> .
8	B	Prompting ép buộc mô hình tuân thủ nguyên tắc Grounded Generation (chỉ trả lời thông tin có trong ngữ cảnh + kèm trích dẫn).
9	B	<code>storage/foundation.json</code> đóng vai trò local cache lưu trữ chỉ mục kèm SHA-256 fingerprint để tối ưu hiệu năng.
10	B	chcp 65001 đổi mã trang Terminal Windows sang UTF-8, giúp hiển thị và xử lý chuỗi tiếng Việt không bị lỗi font.

TÀI LIỆU THAM KHẢO

1. **Lewis, P. et al. (2020).** *"Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks"*. arXiv:2005.11401.
2. **Google Gemini API Documentation:** ai.google.dev/docs
3. **ChromaDB Official Documentation:** docs.trychroma.com
4. **PyMuPDF Documentation:** pymupdf.readthedocs.io